

Full Length Article

Fog-assisted data integrity auditing scheme with deduplication function for cloud storage

Siqi Wu¹, Jinyong Chang^{*}, Yanyan Ji, Yifan Dong, Siyi Wang

School of information and control engineering, Xi'an University of Architecture and Technology, Xi'an, 710055, Shaanxi, P.R. China

ARTICLE INFO

Keywords:

Integrity audit
Data deduplication
Blockchain
Fog node
Cloud storage

ABSTRACT

Cloud storage can not only effectively solve the burden of data owner (DO) storing data locally, but also provide convenient access and sharing services. In cloud storage, deduplication of the same data file can not only save storage resources for cloud server, but also reduce a certain amount of expenses for DO. More importantly, the integrity of stored data determines whether DO continues to enjoy cloud storage services. However, current research tends to separate deduplication from auditing. In this paper, a system is designed by integrating them together based on multiple fog nodes assistance. Specifically, first, multiple fog nodes use threshold technology to generate encryption key and authentication key for the same data file, ensuring the correctness and efficiency of the deduplication function. Second, utilizing a Blockchain maintained by these fog nodes to perform audit tasks guarantees the correctness, transparency, and immutability of the audit process. Finally, the security and performance analysis on the proposed system will be discussed. The results indicate that the proposed system is secure in deduplication and auditing processes, and also has certain advantages in terms of performance.

1. Introduction

With the rapid development of digital technology, both enterprises, governments and individuals have produced a large amount of data. How to store and manage these data has become an important problem for the original data owner (DO). Large scale local storage (Wang et al., 2026) requires not only the purchase of a large number of hardware equipment, but also the provision of professionals for data management, which undoubtedly creates an additional burden on these DOs. Therefore, a new technology is urgently needed to solve the problem of massive data storage. In this setting, cloud storage technology came into being. More specifically, in the cloud storage mode (Alqarni and Fei, 2026), DOs outsource their data to the cloud server (CS), which only needs to charge the corresponding management fees. This can save users the cost of purchasing hardware and personnel management. More importantly, this mode provides convenience for users to access and share data anytime and anywhere. Hence, cloud storage technology has been gradually welcomed by more and more users.

Although cloud storage has many advantages, DOs must worry about whether their outsourced storage data is damaged or lost. This is because after the data is stored in CS, they will inevitably delete the local data to save its cost. In fact, in recent years, cases of accidental loss of user data have occurred from time to time. This situation can be analyzed from internal and external factors. On the one hand, CS

may maliciously delete data that some users do not pay much attention to save their storage resources, and allocate them to other users to absorb more users to rent their storage space (Hertogh et al., 2026); On the other hand, CS may lose or damage the original user's stored data due to external attacks. As a result, how to ensure that DO can effectively know whether the stored data is intact (Yang et al., 2026) is the research content that must be paid attention to in the development of cloud storage technology.

Cloud audit mechanism based on cryptographic technology is just the research direction gradually developed on this basis. In the cloud audit mechanism, the typical implementation method is described as follows. First, DO splits the data file into small blocks and generates authentication tags for them. Then send them together to CS for storage (Wang et al., 2014). Second, when audit is required, DO randomly chooses a certain number of data block indexes and sends them to CS as a challenge message. After receiving it, the CS aggregates the corresponding data file blocks and authentication tags to obtain the audit evidence. Finally, DO can verify the validity of the proof to determine whether the stored data is intact. In fact, in the design of many schemes, in order to further save the computing resources of DO, the audit process can be outsourced to some third-party auditor (TPA), which Aroulanandam et al. (2024) often appears in the system

^{*} Corresponding author.

E-mail address: changjinyong@xauat.edu.cn (J. Chang).

<https://doi.org/10.1016/j.cose.2026.104951>

Received 12 December 2025; Received in revised form 1 April 2026; Accepted 28 April 2026

Available online 30 April 2026

0167-4048/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

models of these schemes. However, the TPA model itself introduces new trust assumptions and potential vulnerabilities. The TPA must be fully trusted to perform audits correctly and to not collude with the cloud server, which reinstates a single point of failure and trust in the system. To move towards a truly decentralized and trust-minimized audit paradigm, recent research has explored replacing the TPA with blockchain technology. For instance, Wang et al. (2020) pointed the use of blockchain to achieve a fully decentralized audit architecture, eliminating the need for a trusted TPA. Similarly, subsequent works like Tian et al. (2022) have leveraged blockchain's tamper-proof nature to secure audit trails in shared storage scenarios. These advances motivate our integration of a fog-node-maintained blockchain to serve not merely as a passive log, but as an active, trustless consistency layer that guarantees the correctness and non-repudiation of the entire audit process.

From the perspective of CS, it needs to face many DOs to complete data storage services. One possible scenario is that different users have the same data. For example, different users in the same enterprise may have duplicate parts on the stored data file. At this time, storing the same files multiple times will inevitably cause a waste of CS storage space. Therefore, only one copy of these same data files is needed to be stored, and these users are allowed to be notified and given certain discounts. In other words, data deduplication for CS is also one of the important aspects of cloud storage research (Zheng et al., 2025). Typically, the data deduplication process often depends on an entity to generate the same authentication key for the same file, so as to ensure that the data file stored in CS is completely consistent with their authentication tag. It can be seen that the fog node needs to face many DOs to complete the tasks of key generation. However, it is not difficult to find that once the fog node is attacked or does not work, then it may leak all user's authentication keys, which will bring immeasurable losses. Hence, in the process of data deduplication (Peng et al., 2025), this security problem becomes more and more prominent.

In this paper, we will comprehensively consider the above factors and design a multi-fog-nodes assisted deduplication mechanism based on existing research work to avoid single point of failure problem. At the same time, a Blockchain, which has the properties of transparent and tamper proof, is built among these fog nodes to complete the audit of cloud storage data integrity. Specifically,

(1) In the deduplication process, DOs are divided into initial DO (IDO) and subsequent DO (SDO). In the key generation stage, based on the owned data files, they interact with these fog nodes and obtain the corresponding encryption and authentication keys. This deterministic method ensures that the same data file owned by different users remains the same even after encryption and authentication. Furthermore, to avoid requiring all fog nodes to participate in each key generation, we designed a process based on threshold secret sharing technology. That is to say, the number of fog nodes participating in generating keys only needs to meet the basic threshold.

(2) During the audit process, use pseudo-random function (PRF) and pseudo-random permutation (PRP) combined with the current audit time to generate corresponding challenge message, which ensures the randomness of the challenge message obtained. After receiving the proof returned by CS, record it on the Blockchain. Of course, any fog node is allowed to verify the validity of the published proof and further record the verification result on the Blockchain.

Our Contributions. To sum up, the contributions of this paper can be summarized as follows.

- (1) A key generation mechanism based on multiple fog nodes will be designed to ensure that each identical data file is recognized by CS and thus achieve correct deduplication. Meanwhile, it does not require the participation of all fog nodes.
- (2) An integrity auditing scheme based on Blockchain, maintained by these fog nodes, is proposed, which not only ensures the correctness of auditing, but also achieves transparency and immutability in the auditing process.

- (3) A detailed security analysis, including the security of the audit process and the deduplication process, and performance analysis will be given on the proposed system.

Related Works.

Cloud storage security mechanisms constitute the foundational framework for establishing trusted cloud services. In 2007, Ateniese et al. (2007) made a seminal contribution by proposing the Provable Data Possession (PDP) scheme. This approach innovatively employs probabilistic sampling techniques to achieve efficient remote verification with constant communication overhead, a critical advantage for large-scale cloud storage systems. Concurrently, Juels and Kaliski (2007) introduced Proofs of Retrievability (POR), providing a complementary solution that not only verifies data integrity but also ensures data recoverability. Armbrust et al. (2010) pioneered a systematic framework characterizing cloud storage architectures and their core features. While this model significantly enhances operational efficiency, it inherently transfers data control from end-users to service providers, thereby introducing unprecedented security vulnerabilities. Furthermore, these foundational contributions share fundamental limitations: their reliance on static data assumptions and fully trusted third parties demonstrates a significant deviation from real-world application requirements, revealing a pronounced theoretical-practical gap.

To address the inherent trust issues in third-party auditing, the research community has made progressive innovations. Rizvi et al. (2015) made the first breakthrough by establishing a novel TPA-based verification mechanism through the integration of dynamic session keys and service-level agreements, effectively mitigating potential insider threats. Building upon this foundation, Wang et al. (2020) achieved a paradigm shift by leveraging blockchain technology to replace traditional TPAs, thereby realizing a fully decentralized auditing architecture through distributed ledger mechanisms. Further refining the theoretical framework, Swapnali and Sangita (2016) established a comprehensive cloud auditing model encompassing three key entities (data owners, cloud service providers, and auditors), which optimizes task allocation to balance security and efficiency. As application demands evolved, auditing capabilities underwent continuous expansion. Tian et al. (2017) introduced an advanced dynamic hashing structure, enabling the system to efficiently handle data updates and finally overcoming the long-standing "static assumption" constraint. Concurrently, Liu (2025) developed a novel solution incorporating lattice-based cryptography and Bloom filters, which not only enhances privacy protection but also provides quantum-resistant security properties. For emerging network environments, Yu and Jun (2024) specifically addressed the challenges of multi-copy integrity verification in 5G networks, proposing an innovative scheme that supports efficient batch auditing while maintaining robust data consistency. Xu et al. (2026) proposed the MPA scheme, a lightweight publicly verifiable auditing mechanism designed for decentralized storage systems. Although the MPA scheme achieves considerable improvements in auditing efficiency and transparency, it neither supports data deduplication nor provides a decentralized key management framework. Instead, its key generation process still relies on centralized entities.

Parallel to auditing advancements, data deduplication technologies have undergone significant security enhancements. The foundational work by Douceur et al. (2002) introduced convergent encryption (CE), establishing the first secure framework for encrypted deduplication by cryptographically binding encryption keys to file content. Bellare et al. (2013) subsequently formalized this approach through the rigorous mathematical framework of Message-Locked Encryption (MLE). However, Halevi et al. (2011) identified a critical attack vector where adversaries could reconstruct entire files using only hash values, leading to the development of Proofs of Ownership (PoWs) based on Merkle tree structures. Simultaneously, Ha et al. (2025) discovered that deterministic deduplication responses could leak sensitive metadata, prompting the invention of randomized response techniques to achieve better

security-privacy tradeoffs. With the emergence of edge computing, Luo et al. (2025) redesigned the deduplication architecture for distributed edge environments, solving the compatibility issues of traditional centralized approaches. He et al. (2026) proposed an intelligent compression framework to help users determine whether a dataset is suitable for deduplication or compression operations. However, their work neither investigates secure deduplication schemes integrated with integrity auditing nor addresses the inherent key management challenges in encrypted deduplication systems.

To integrate auditing with deduplication, Yuan and Yu (2013) pioneered this direction by developing a novel scheme combining polynomial-based authentication tags with homomorphic linear authenticators, enabling simultaneous deduplication of both files and their verification tags. However, the centralized nature of this approach introduces single-point-of-failure risks. Tian et al. (2022) subsequently proposed a blockchain-based alternative employing a dual-server model, though the solution faces scalability challenges in multi-user scenarios due to excessive key storage requirements. A significant breakthrough came from Miao et al. (2023), who applied identity-based broadcast encryption to achieve key deduplication, reducing user-side storage complexity from $O(F)$ to $O(1)$ while eliminating dependency on key servers. Zhang et al. (2025) further enhanced this approach by integrating blockchain auditing with identity-based encryption, providing additional protection for file tags. While several recent schemes integrate blockchain and fog computing for secure deduplication and auditing, their application scenarios and architectural assumptions differ significantly from ours. For instance, Gu et al. (2024) focus on VANET scenarios and design a consensus mechanism tailored specifically for vehicles. Their key generation mechanism is vulnerable to brute-force attacks. In their auditing phase, fog nodes merely forward signals without participating in the auditing process. Moreover, their system may introduce additional latency when deployed in general fog-cloud environments. Zhang et al. (2024) target IoT environments with key-server dependency, which centralizes key management and may become a bottleneck. Therefore, although existing integrated systems have two major functions, they still have unresolved deficiencies. In contrast, our scheme is designed for general-purpose cloud storage systems with distributed fog nodes, emphasizing threshold-based key generation without key servers and a lightweight blockchain maintained solely by fog nodes.

Organizations. The remaining sections of this paper are structured as follows. First, Section 2 introduces the basic notations and related knowledge involved in our scheme. In Section 3, we elaborate on the system model and security model. For Section 4, we describe the specific details of the scheme. Section 5 provides correctness and security analysis of the scheme. Then, Section 6 presents the theoretical analysis and experimental results of our scheme. Finally, the paper concludes with a summary.

2. Preliminaries

2.1. Notations

Symbols and their corresponding definitions are listed in Table 1.

2.2. Bilinear map

Let G_1 and G_2 be two multiplicative cyclic groups of prime order q , with g being the generator of G_1 . Suppose $e : G_1 \times G_1 \rightarrow G_2$ is a bilinear map that satisfies the following properties:

- **Bilinearity.** For all $P_1, P_2 \in G_1$, $a, b \in \mathbb{Z}_p$, it holds $e(P_1^a, P_2^b) = e(P_1, P_2)^{ab}$.
- **Non-degeneracy.** For all $P_1, P_2 \in G_1$, $a, b \in \mathbb{Z}_p$, there must exist $e(P_1, P_2) \neq 1$.
- **Computability.** For all elements $P_1, P_2 \in G_1$, $e(P_1, P_2)$ must can be calculated.

Table 1
Notations.

Symbols	Definitions
IDO	Initial data owner
SDO	Subsequent data owner
FN_s	Fog nodes
CS	Cloud server
G_1, G_2	Two multiplicative cyclic groups
g	Generator of G_1
PRP, PRF	Pseudorandom functions
H, H_1	Hash function
Θ	Security parameter
$\{g_1, g_2, \dots, g_m\}$	Elements in G_1
(sk_u, pk_u)	Secret-public key pair for FN_s
s	Shared secret value of fog nodes
F	Original data file F
(sk, pk)	Authentication key pair for F
K	Encryption key of F
ID	The file label of F
CM	Ciphertext set of F
U	Authenticated tag set of CM
$chal_1$	PoW-challenge message
$proof_1$	PoW-proof of SDO
$chal_a$	Audit-challenge message
$proof_a$	Audit-integrity proof
T	Current timestamp
PPT	Probabilistic polynomial time
PoW	Proof of ownership
CDH	Computational Diffie–Hellman
DL	Discrete logarithm

2.3. Difficulty assumptions

The discrete logarithm problem is a foundational computational challenge in cryptography and number theory, underpinning the security of numerous public-key cryptosystems.

- **Discrete Logarithm (DL) Assumption.** Let G_1 be a cyclic group of prime order p with generator g . For $h = g^x \in G_1$ where $x \in \mathbb{Z}_p$ is uniformly random, no probabilistic polynomial-time (PPT) algorithm can compute x with non-negligible probability.
- **Computational Diffie–Hellman (CDH) Assumption.** Let G_1 be a cyclic group of prime order p with generator g . For uniformly random $a, b \in \mathbb{Z}_p$, given (g, g^a, g^b) , no PPT algorithm can compute $g^{ab} \in G_1$ with non-negligible probability.

2.4. Proof of ownership

Proof of Ownership (PoW) is a cryptographic protocol enabling data owners to prove possession of a complete data copy to storage providers without transmitting the entire dataset. In deduplication scenarios, PoW employs lightweight challenge-response mechanisms to ensure only legitimate owners can authorize deletions, mitigating malicious resource occupation via hash-spoofing or metadata forgery. Its core framework includes.

- $Pow.chal(Para) \rightarrow (chal)$. Executed by the challenger to generate a challenge $chal$ using public parameters $Para$.
- $Pow.proof(Para, chal) \rightarrow (proof)$. Executed by the data owner to generate a $proof$ using $Para$ and $chal$, which is then sent to the challenger.
- $Pow.verify(proof) \rightarrow (result)$. Executed by the challenger to validate $proof$ and output $result$, determining the success of the owner's challenge.

2.5. Threshold secret sharing

Shamir's threshold secret-sharing scheme enables secure distributed secret storage and management by splitting a secret S into n shares among participants, with a threshold t ($1 \leq t \leq n$) satisfying:

- S can be uniquely reconstructed only with at least t shares.
- Any subset of fewer than t shares reveals no information about S .

It is mainly applied in scenarios such as key management and distributed systems. Through the mechanism of “risk decentralization and collective control”, it strikes a balance between security and availability, and stands as one of the fundamental technologies in the field of distributed security.

3. System and security models

In this section, we will introduce the system and security models as well as the design goals for the proposed scheme.

3.1. System model

In our proposed system, there are three kinds of entities: data owners (DOs), fog nodes (FNs) and cloud server (CS), in which DOs are divided into the initial owner (IDO) and the subsequent one (SDO). The interactions of these entities are shown in Fig. 1.

- **Data owners:** In order to save local storage space, DOs first obtain the key-slices from FNs, then encrypt and authenticate the original data, and store it on the cloud server. In fact, only IDO needs to perform data encryption and authentication processes. For SDO, since CS has already stored the authenticated ciphertext from IDO, it only needs to prove the ownership of ciphertext.
- **Fog nodes:** FN has two functions: On the one hand, it assists DO in generating key-slice, and on the other hand, it utilizes its internally built Blockchain to perform integrity audit on the stored data. More concretely, in key generation mode, upon receiving a request from DO, each FN calculates and returns a key-slice. In audit mode, any FN is allowed to generate a challenge message, and transmit it to CS. After receiving the returned proof, record it on the ledger of the Blockchain. The transparency and immutability of Blockchain ensure that the audit process has the same properties.
- **Cloud server:** CS has a massive storage space that can provide data storage services for multiple DOs. At the same time, it also accepts challenge requests initiated by FN, calculates and returns the corresponding proof. Once data loss or damage is detected, it will compensate DO with certain fees. In addition, in order to achieve deduplication of the same data, it checks ownership of SDO to ensure that SDO has the same file.

In order to ensure that the designed system can achieve secure deduplication and integrity auditing, the needed algorithms are introduced below.

- $Setup(\Theta) \rightarrow (Para)$: Given a security parameter Θ , this algorithm outputs the public parameter $Para$ for the system.
- $(s, \{sk_u, pk_u\}_{u=1}^{\ell}) \leftarrow KeySetup(FN_1, FN_2, \dots, FN_{\ell})$: This is a sub-protocol running by ℓ fog nodes, which will output a shared secret value s and a secret-public key pair (sk_u, pk_u) for the u th fog node FN_u .
- $(K, sk, pk) \leftarrow KeyGen(F, \{sk_u, pk_u\}_{u=1}^{\ell})$: This algorithm will generate the encryption key K and the authentication key pair (sk, pk) for some data file F . That is, given the input $(F, \{sk_u, pk_u\}_{u=1}^{\ell})$, it outputs the tuple (K, sk, pk) .
- $(ID, CM, U) \leftarrow AuthGen((K, sk, pk), F)$: Based on the generated keys from the fog nodes, this algorithm computes and outputs the encrypted file CM together with the corresponding authentication tag U . Here, ID denotes the filename of the original data file F .
- $chal_1 \leftarrow Pow.chal$: This algorithm generates a challenge message $chal_1$ for CS to check the proof of ownership of SDO for the same file.

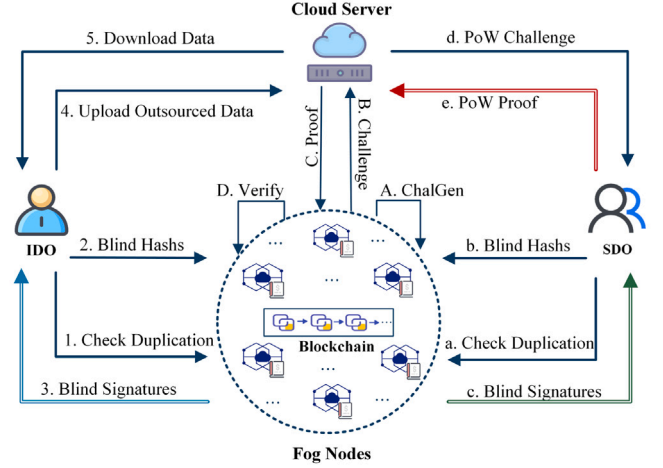


Fig. 1. System model of our scheme.

- $proof_1 \leftarrow Pow.proof(chal_1)$: Based on the received $chal_1$, this algorithm will generate a proof $proof_1$ for SDO, which is used to prove the ownership of data file.
- $1/0 \leftarrow Pow.verify(chal_1, proof_1)$: This algorithm will verify the validity of $proof_1$. It outputs 1 if it passes the verification. Else, output 0.
- $chal_a \leftarrow Audit.chal$: This algorithm will generate a challenge message $chal_a$ in the auditing phase, which is given to CS.
- $proof_a \leftarrow Audit.proof(chal_a, ID, CM, U)$: It is used to generate a proof $proof_a$ for CS when given the challenge message $chal_a$ and the stored (ID, CM, U) .
- $1/0 \leftarrow Audit.verify(chal_a, proof_a)$: This algorithm is run by fog nodes to check the validity of $proof_a$. In particular, after receiving the returned $proof_a$, it will output 1 if the proof passes the verification. Else, output 0.

3.2. Security model

To secure our proposed scheme, we identify trust assumptions, potentially adversarial entities and formulates two games to guarantee the security of the system during data auditing and deduplication processes.

- **Fog nodes.** We assume that at least t out of ℓ fog nodes are honest and follow the protocol correctly. These nodes do not collude to reveal secret key materials. The threshold value t is a system parameter determined during the initialization phase by the system administrator based on the total number of fog nodes ℓ and the desired trade-off between security and availability. Specifically, t is set as a public parameter and written into the system configuration, balancing three key factors: collusion resistance, fault tolerance, and communication overhead.
- **Half-trusted Cloud Server (CS).** CS are obligated to ensure data integrity; however, a malicious cloud may intentionally delete infrequently accessed files to conserve storage resources, in turn breaching service agreements.
- **Malicious Subsequent Data Owner (SDO).** A malicious SDO may lack the complete original data yet attempt to successfully pass the cloud's PoW challenge to gain ownership of the data.

Based on the two threats above, we define security models via the following two games, respectively, to ensure the security of auditing and deduplication phases.

Game 1: We suppose a security game 1 between challenger B and adversary A , where A refers a half-trusted CS and B is the game initiator.

- **Initialization.** The challenger B executes *Setup* to generates the public parameters $Para$ sends to A . Subsequently, it executes the *KeyGen* and *AuthGen* algorithms to generate $\{ID, CM\}$ and sends to the adversary A .
- **Query.** A selects a block c_i from set CM , and requests B to generate and return the corresponding authentication tag σ_i . This process allows A to initiate multiple queries.
- **Challenge.** B issues a random, unqueried challenge $chal$, sending it to A and requiring A to generate an integrity *proof* based on $chal$.
- **Forgery.** A forges an integrity proof from $chal$ and submits it to B ; A succeeds in its attack if the validation passes.

Definition 1. In arbitrary probabilistic polynomial time (*PPT*), a half-honest CS can pass the authentication with negligible probability if it does not hold the correct and complete data.

Game 2: We suppose a security game 2 between a challenger B and a malicious adversary A for malicious SDO to build a security model for deduplication.

- **Initialization.** B executes algorithm *Setup* to generates $Para$ and sends it to adversary A .
- **Query.** A selects file F and sends it to B , which executes algorithm *KeyGen* to generate an encryption key K and returns it. Concurrently, A sends file block data to B via multiple queries, requesting the corresponding signatures σ_i .
- **Challenge.** B sends A a challenge message $chal$ that does not include previously queried content and requires the corresponding *proof* in return.
- **Forgery.** A forges an integrity proof and sends to B . If the verification succeeds, A wins.

Definition 2. In arbitrary probabilistic polynomial time (*PPT*), our scheme can securely resist SDO from obtaining file ownership by forgoing evidence.

3.3. Design goals

In this section, we establish the design goals of our scheme based on the security model.

- **Deduplication correctness.** During the deduplication process, it is essential to ensure that identical data files from different data owners (DOs), after encryption and authentication, can be correctly identified by the cloud server (CS). This requires identical data to be encrypted with the same key, thus preventing false negatives or false positives.
- **Confidentiality.** In the proposed scheme, sensitive data must be protected from unauthorized access. Encryption keys and authentication information shall be securely generated and managed, ensuring that cloud servers, fog nodes, or malicious third parties cannot decrypt or infer ciphertext. Meanwhile, file content and ownership details remain protected during deduplication and auditing.
- **Integrity.** In our proposed scheme, data integrity must be ensured throughout the process. The data auditing protocol should accurately detect malicious behaviors of the cloud server (CS). Furthermore, audit results should be verifiable and trustworthy, enabling data owners (DOs) to confirm the integrity of their data.
- **Efficiency.** Our scheme adopts distributed fog nodes for user encryption key generation, ensuring high security and favorable performance during deduplication. Additionally, computational and communication overheads are minimized in both auditing and deduplication phases, guaranteeing scalability for large-scale distributed data storage scenarios.

4. Our proposed construction

In this section, we will give the concrete algorithms of the proposed system. For clarity, we divided these algorithms into four phases: System initialization, initial data upload, subsequent data upload, and integrity auditing, which are introduced as follows.

4.1. System initialization

The initialization phase of the system includes the algorithm *Setup* and the sub-protocol *KeySetup*.

- $Para \leftarrow Setup(\Theta)$: Given the security parameter Θ , this algorithm runs as follows. First, choose two multiplicative groups G_1, G_2 with the same prime order p ($|p| = \Theta$). Let g be a generator of G_1 , and $H : \{0, 1\}^* \rightarrow G_1, H_1 : \{0, 1\}^* \rightarrow Z_p$ be two hash functions. Define $e : G_1 \times G_1 \rightarrow G_2$ as a bilinear map and randomly choose elements $g_1, g_2, \dots, g_m \in G_1$. Then define $\pi_1 : Z_p \times [n] \rightarrow [n]$ and $\pi_2 : Z_p \times [n] \rightarrow Z_p$ as PRP and PRF, respectively. Finally, the public parameter is set as

$$Para = \{G_1, G_2, e, g, \{g_1, g_2, \dots, g_m\}, H, H_1, \pi_1, \pi_2\}.$$

Without loss of generality, assume that $Para$ is implicitly included in the inputs of other algorithms.

- $(s, \{sk_u, pk_u\}_{u=1}^\ell) \leftarrow KeySetup(FN_1, FN_2, \dots, FN_\ell)$: The u th fog node FN_u generates a random polynomial of degree $t-1$ over Z_p :

$$f_u(x) = a_{u,0} + a_{u,1}x + \dots + a_{u,t-1}x^{t-1},$$

and broadcasts $\{g^{a_{u,0}}, g^{a_{u,1}}, \dots, g^{a_{u,t-1}}\}$ and $f_u(\omega)$ to other nodes. Then compute its private key as $sk_u = \sum_{\omega=1}^n f_\omega(u) \in Z_p$ and public key as $pk_u = g^{sk_u}$. Finally, the shared secret key for the ℓ fog nodes is defined as $s = \sum_{u=1}^n a_{u,0}$.

4.2. Key generation

- $(K, sk, pk) \leftarrow KeyGen(F, \{sk_u, pk_u\}_{u=1}^\ell)$: When DO wants to upload data file F to CS, it first divides the file F into n blocks $\{m_i\}_{i=1}^n$ and computes

$$\{H(m_i)\}_{i=1}^n, FL = H(H(m_1) \| H(m_2) \| \dots \| H(m_n)).$$

Then send $\{ID_{DO} \| H(m_1) \| \dots \| H(m_n) \| FL\}$ to all the fog nodes. For u th fog node FN_u , it computes $s_i^{(u)} = (H(m_i) \| i)^{sk_u}$ (for $1 \leq i \leq n$) and $s^{(u)} = H(FL)^{sk_u}$. Finally, return them to DO. Note that, only need to collect values from (at least) t fog nodes. Typically, the threshold t can be set as $t = \lceil \ell/2 \rceil + 1$, which ensures that even if up to $\ell - t$ fog nodes are compromised or offline, the system can still generate correct keys. DO can compute the components:

$$k_i = \prod_{u=1}^t \left(s_i^{(u)} \right)^{\lambda_u},$$

where $\lambda_u = \prod_{j \in [t], j \neq u} \frac{j}{j-u}$. Finally, set $K = H_1(\prod_{i=1}^\ell k_i)$, $sk = H_1(s^{(u)})$, and $pk = g^{sk}$. As illustrated in Fig. 2, it can be clearly observed that only fog nodes and data owners act as the participating entities in the key generation phase.

4.3. Data uploading

(1) Initial Uploading

For the initial data owner IDO, let (K, sk, pk) be the returned key from the fog nodes. Then it encrypt the file F with K based on any symmetric encryption algorithm *Encrypt* and authenticate the blocks of the ciphertext by using sk . More concretely, it runs the following algorithm. The process is shown in Fig. 2 and described as follows.

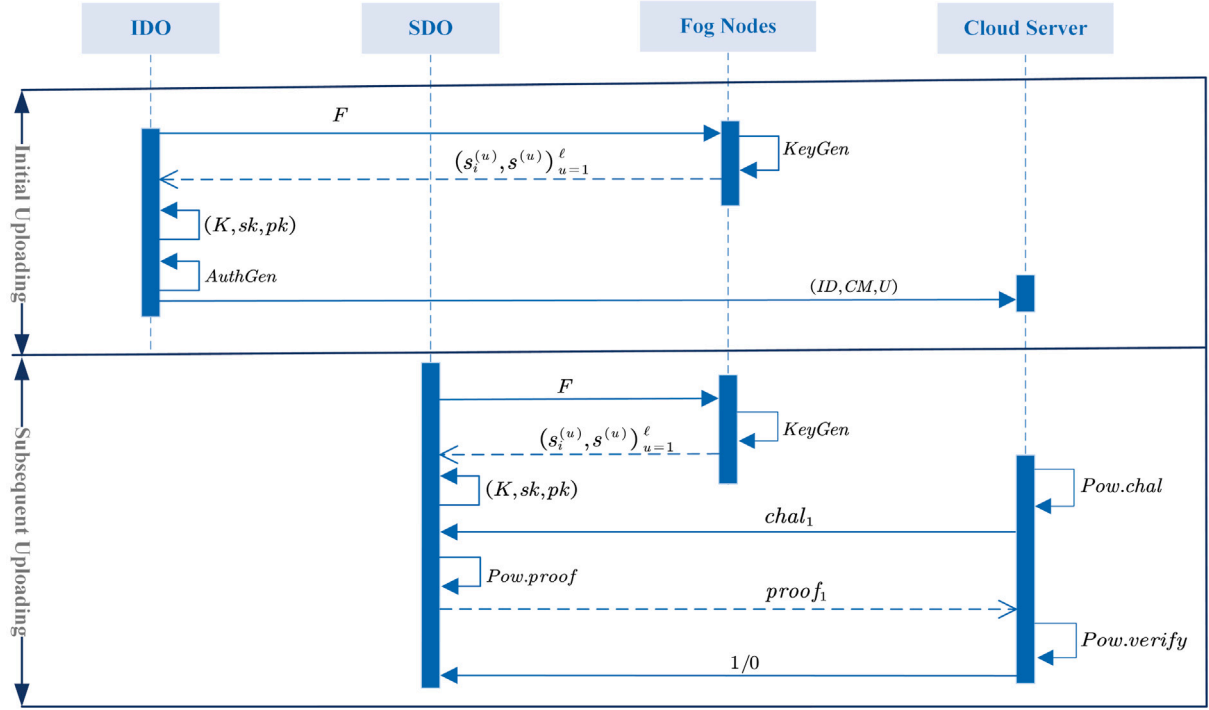


Fig. 2. The workflows of data uploading.

- $(ID, CM, U) \leftarrow AuthGen((K, sk, pk), F)$: First, define $ID = FL$ as the filename of F . For the i th block m_i , compute $c_i = Encrypt(K, m_i)$. Then parse c_i as m segments $c_{i,1}, c_{i,2}, \dots, c_{i,m}$. For $1 \leq i \leq n$, define $\beta_i = H(ID||i)$ and compute the corresponding tag as

$$\sigma_i = (\beta_i \cdot g_1^{c_{i,1}} g_2^{c_{i,2}} \dots g_m^{c_{i,m}})^{sk}.$$

Define $CM = (c_1, c_2, \dots, c_n)$, $U = (\sigma_1, \sigma_2, \dots, \sigma_n)$ as the ciphertext and authenticated tag, respectively, and output (ID, CM, U) , which will be transmitted to CS for storing.

(2) Subsequent Uploading

For the subsequent data owner SDO, it needs to interact with CS to jointly complete the proof of ownership for the same file F , which can be divided into the following two steps.

- $chal_1 \leftarrow Pow.chal$: The CS first randomly chooses $r_1, r_2 \in Z_p, k \in [n]$ and sends $chal_1 = (r_1, r_2, k)$ to SDO as the challenge message.
- $proof_1 \leftarrow Pow.proof(chal_1)$: The SDO will compute

$$a_i = \pi_1(r_1, k) \in [n], b_i = \pi_2(r_2, k) \in Z_p.$$

Meanwhile, it needs to generate the ciphertext for the block m_{a_i} by computing $c_{a_i} = Encrypt(K, m_{a_i})$, where K is the symmetric key obtained from fog nodes. Define $v_j = b_i \cdot c_{a_{i,j}}$ and return $proof_1 = (c_{a_i}, v_i)$ to CS.

- $1/0 \leftarrow Pow.verify(chal_1, proof_1)$: The CS verifies the ownership of the original data file by checking

$$e(\sigma_{a_i}^{b_i}, g) = e(\beta_{a_i}^{b_i} \cdot g_j^{v_j}, pk),$$

where $(\sigma_{a_i}, \beta_{a_i})$ is the auxiliary information stored by IDO for the same data file. Then SDO passes the verification of ownership if and only if the above equation is correct.

4.4. Data auditing

In this phase, a fog node is responsible for performing the auditing task, and the process is recorded on the ledger of Blockchain, which

is constructed among the ℓ fog nodes. In particular, once entering data auditing state, these ℓ fog nodes runs a lightweight consensus protocol (a variant of Practical Byzantine Fault Tolerance adapted for fog networks) to elect a leader node FN_w for each audit round. The consensus ensures that even if up to $f = (\ell - 1)/3$ fog nodes are Byzantine or fail, the system can still agree on a legitimate auditor. This design provides fault tolerance against node failures and malicious behavior while maintaining low communication overhead.

- $chal_a \leftarrow Audit.chal$: Let T be the current timestamp and randomly choose a number $z \in [n]$. Compute

$$a_i = \pi_1(H_1(T), z) \in [n], b_i = \pi_2(H_2(T), z) \in Z_p.$$

Then denote by $chal_a = (z, a_i, b_i)$ the challenge message given to CS.

- $proof_a \leftarrow Audit.proof(chal_a, ID, CM, U)$: After receiving the message $chal_a = (z, a_i, b_i)$, CS computes the proof as follows based on the stored (ID, CM, U) . For $1 \leq j \leq m$, calculate

$$v_j = \sum_{i=1}^z b_i \cdot c_{a_{i,j}}, \quad \rho = \prod_{i=1}^z \sigma_{a_i}^{b_i},$$

and define $proof_a = (v_1, v_2, \dots, v_m, \rho)$. Finally, return it to FN_w , who will record $(chal_a, proof_a)$ on the ledger of the Blockchain.

- $1/0 \leftarrow Audit.verify(chal_a, proof_a)$: Any fog node is allowed to verify the validity of the returned $proof_a$. More concretely, any node is allowed to check the validity by computing the following equation:

$$e(\rho, g) = e\left(\prod_{i=1}^z \beta_{a_i}^{b_i} \cdot \prod_{j=1}^m g_j^{v_j}, pk\right). \quad (1)$$

If it is correct, output 1. Otherwise, output 0.

5. Correctness and security analyses

In this section, we will present the correctness and security analysis of our proposed scheme.

5.1. Correctness

Given the honest-entity assumption that the data owners, cloud server, and fog nodes strictly adhere to the protocol, we prove the model's correctness via formal analysis. The verification process is as follows:

$$\begin{aligned} e\left(\prod_{i=1}^z \sigma_{a_i}^{b_i}, g\right) &= e\left(\prod_{i=1}^z \left(\beta_{a_i} \cdot g_1^{c_{a_i1}} g_2^{c_{a_i2}} \dots g_m^{c_{a_im}}\right)^{sk \cdot b_i}, g\right) \\ &= e\left(\prod_{i=1}^z \left(\beta_{a_i} \cdot g_1^{c_{a_i1}} g_2^{c_{a_i2}} \dots g_m^{c_{a_im}}\right)^{b_i}, g^{sk}\right) \\ &= e\left(\prod_{i=1}^z \left(\beta_{a_i}^{b_i} \cdot g_1^{b_i c_{a_i1}} g_2^{b_i c_{a_i2}} \dots g_m^{b_i c_{a_im}}\right), g^{sk}\right) \\ &= e\left(\prod_{i=1}^z \beta_{a_i}^{b_i} \left(\prod_{j=1}^m g_j^{\sum_{i=1}^z b_i c_{a_ij}}\right), pk\right) \\ &= e\left(\prod_{i=1}^z \beta_{a_i}^{b_i} \cdot \prod_{j=1}^m g_j^{v_j}, pk\right), \end{aligned}$$

when $v_j = \sum_{i=1}^z b_i c_{a_ij}$, $\rho = \prod_{i=1}^z \sigma_{a_i}^{b_i}$, we can obtain

$$e(\rho, g) = e\left(\prod_{i=1}^z \beta_{a_i}^{b_i} \cdot \prod_{j=1}^m g_j^{v_j}, pk\right).$$

5.2. Security analysis

In this section, we will present the security analysis on data auditing and deduplication in our scheme based on the security model.

Theorem 1. Based on the intractability of the Diffie–Hellman (CDH) problem and the intractability of the discrete logarithm problem (DL), the adversary $\mathcal{A}$ cannot pass the verification by forging cipher-text CM or authenticator U in the audit verification phase.

Proof. The proof of Theorem 1 relies on Game 1 (as specified in Definition 1) and involves interaction between challenger $\mathcal{B}$ and adversary $\mathcal{A}$. The game is described as follows:

- **Game 3 :** It differs from Game 1 in that $\mathcal{B}$ and $\mathcal{A}$ engage in multiple queries and challenges, with $\mathcal{B}$ maintaining relevant records. If $\mathcal{A}$ responds to a challenge with a result inconsistent with $\prod_{i=1}^z \rho_{a_i}^{b_i}$, $\mathcal{B}$ declares $\mathcal{A}$ has failed and terminates the game.
- **Analysis:** Based on the CDH problem assumption, we construct a simulator. Given the tuple $(g, g^a, g^b) \in G_1^3$ with unknown $a, b \in \mathbb{Z}_q$, the simulator aims to output g^{ab} . We define the public parameters as follows: $pk = g^{sk} = g^a$, $\beta_{a_i} = g^a \cdot \prod_{j=1}^m g_j^{\delta_j}$ where $\delta_j \in \mathbb{Z}_p$ is a random number, $v_j = \delta_j$ and $u = g^a h^\beta$. When adversary $\mathcal{A}$ requests a signature on attribute set a_i , simulator $\mathcal{B}$ generates a valid signature ρ_{a_i} . From this, the aggregated valid authenticator is derived as:

$$\prod_{i=1}^z \rho_{a_i}^{b_i} = g^{a^2 \sum_{i=1}^z b_i} \cdot \prod_{j=1}^m g_j^{2a \delta_j \sum_{i=1}^z b_i}.$$

Suppose $\mathcal{A}$ successfully forges $\{\rho_{a_i}^*\}_{i=1}^z$ and the corresponding data block satisfying $v_j^* \neq v_j$. The verification formulas for the forged signatures are as follows:

$$e\left(\prod_{i=1}^z \left(\rho_{a_i}^*\right)^{b_i}, g\right) = e\left(\prod_{i=1}^z \beta_{a_i}^{b_i} \cdot \prod_{j=1}^m g_j^{\sum_{i=1}^z b_i v_j^*}, pk\right).$$

Based on our proposal, the following equation holds:

$$e(\rho, g) = e\left(\prod_{i=1}^z \beta_{a_i}^{b_i} \cdot \prod_{j=1}^m g_j^{v_j}, pk\right).$$

If $\mathcal{A}$ succeeds in forging the $proof_a^* = (\rho^*, v_j^*)$, we can obtain the following equation:

$$e(\rho^*, g) = e\left(\prod_{i=1}^z \beta_{a_i}^{b_i} \cdot \prod_{j=1}^m g_j^{v_j^*}, pk\right).$$

We clearly have $v_j \neq v_j^*$; otherwise, $\rho \neq \rho^*$, which results in a contradiction. By dividing the verification formula of a valid signature by that of a forged signature:

$$\frac{e\left(\prod_{i=1}^z \left(\rho_{a_i}^*\right)^{b_i}, g\right)}{e\left(\prod_{i=1}^z \beta_{a_i}^{b_i}, g\right)} = \frac{e\left(g^{a^2 \sum b_i} \cdot \prod g_j^{sk \sum b_i (\delta_j + v_j^*)}, g\right)}{e\left(g^{a^2 \sum b_i} \cdot \prod g_j^{2a \delta_j \sum b_i}, g\right)}.$$

Simplifying the above equation and isolating the key term $\prod_{j=1}^m e\left(g_j^{sk \sum b_i (v_j^* - \delta_j)}, g\right)$, with the simulator setting $v_j^* = \delta_j + \theta_j \cdot b$, we obtain the following equation:

$$\prod_{j=1}^m e\left(g_j^{sk \cdot b \sum b_i \theta_j}, g\right) = e\left(\prod_{j=1}^m g_j^{\sum b_i \theta_j}, g^{sk \cdot b}\right).$$

When $\prod_{j=1}^m g_j^{\sum b_i \theta_j} = g^a$, we can extract the following equation for the CDH solution:

$$e(g^a, g^{sk \cdot b}) = e(g^{ab}, g^{sk}).$$

We can deduce that if the adversary succeeds in forging a signature, the simulator can extract g^{ab} , which contradicts the CDH assumption. The adversary's advantage $\text{Adv}_{\mathcal{A}}$ is thus bounded by the CDH advantage Adv_{CDH} , expressed as $\text{Adv}_{\mathcal{A}} \leq \text{Adv}_{\text{CDH}}$. Since the CDH problem is intractable in polynomial time, the adversary cannot succeed in the attack with non-negligible probability.

- **Game 4 :** The Game 4 is similar to Game 3. If $\mathcal{A}$ responds to the challenge with a result different from v_j , challenger $\mathcal{B}$ declares the challenge failed and terminates the game.
- **Analysis:** According to our scheme, a valid $proof_a = (\rho, v_j)$ is generated with the following equation:

$$e(\rho, g) = e\left(\prod_{i=1}^z \beta_{a_i}^{b_i} \cdot \prod_{j=1}^m g_j^{v_j}, pk\right).$$

Furthermore, if $\mathcal{A}$ successfully forges $proof_a^* = (\rho^*, v_j^*)$ such that $proof_a \neq proof_a^*$, we obtain the following equation:

$$e(\rho^*, g) = e\left(\prod_{i=1}^z \beta_{a_i}^{b_i} \cdot \prod_{j=1}^m g_j^{v_j^*}, pk\right).$$

We construct a simulator to solve the DL problem, described as follows. For each attribute a_i and dimension j , we randomly select $\alpha_i, \beta_i \in \mathbb{Z}_p$, and define $v_j = \alpha_i + \beta_i \cdot x$ and $pk = g^{sk}$. After generating the attribute parameters $\beta_{a_i} = g^{d_i} \cdot \prod_{j=1}^m g_j^{-\alpha_i}$, where d_i is a random number and $d_i \in \mathbb{Z}_p$. We construct a simulator to generate the following equation:

$$\rho_{a_i} = \left(\beta_{a_i} \cdot \prod_{j=1}^m g_j^{v_j}\right)^{sk}.$$

Substituting $v_j = \alpha_i + \beta_i \cdot x$ and $\beta_{a_i} = g^{d_i} \cdot \prod_{j=1}^m g_j^{-\alpha_i}$ to generate following equation:

$$\rho_{a_i} = \left(g^{d_i} \cdot \prod_{j=1}^m g_j^{-\alpha_i} \cdot \prod_{j=1}^m g_j^{\alpha_i + \beta_i x}\right)^{sk} = \left(g^{d_i} \cdot \prod_{j=1}^m g_j^{\beta_i x}\right)^{sk}.$$

Next, the adversary $\mathcal{A}$ performs the forged of labels. Suppose $\mathcal{A}$ successfully forges, in some instance, the data block $v_j^* \neq v_j$ corresponding to set $\{\rho_{a_i}^*\}_{i=1}^z$; we then obtain the following equation:

$$e\left(\prod_{i=1}^z \left(\rho_{a_i}^*\right)^{b_i}, g\right) = e\left(\prod_{i=1}^z \beta_{a_i}^{b_i} \cdot \prod_{j=1}^m g_j^{\sum_{i=1}^z b_i v_j^*}, pk\right)$$

Table 2
Functional comparison with related schemes.

Scheme	Deduplication	Integrity auditing	Fog-assisted key generation	Blockchain
Scheme (Ha et al., 2025)	Y	N	N	N
Scheme (Tian et al., 2022)	Y	Y	N	Y
Scheme (Zhang et al., 2025)	Y	Y	N	Y
Scheme (Gu et al., 2024)	Y	Y	N	Y
Scheme (Zhang et al., 2024)	Y	N	Y	Y
Our scheme	Y	Y	Y	Y

For $\mathcal{A}$ to succeed in forgery, there must exist at least one $v_j^* \neq v_j$; otherwise, a contradiction arises. Dividing the verification formula of a forged signature by that of a valid signature yields:

$$\frac{e\left(\prod (\rho_{a_i}^*)^{b_i}, g\right)}{e\left(\prod \rho_{a_i}^{b_i}, g\right)} = \frac{e\left(\prod \beta_{a_i}^{b_i} \cdot \prod g_j^{\sum b_i v_j^*}, pk\right)}{e\left(\prod \beta_{a_i}^{b_i} \cdot \prod g_j^{\sum b_i v_j}, pk\right)}.$$

Given $\Delta v_j = v_j^* - v_j \neq 0$, substituting into the equation $v_j = \alpha_i + \beta_i x$, we have $\Delta v_j = \beta_i \cdot (x^* - x)$, where x^* is the unknown value forged by the adversary. Expanding further we get the following equation:

$$e\left(\prod_{j=1}^m g_j^{\sum b_i \beta_i (x^* - x)}, g^{sk}\right) = e\left(\prod_{j=1}^m g_j^{sk \cdot \sum b_i \beta_i (x^* - x)}, g\right).$$

If $\mathcal{A}$ succeeds in forging, there exists a nonzero term $\sum b_i \beta_i (x^* - x)$ and we have $e(h^{\beta \cdot sk \cdot (x^* - x)}, g) = e(g, g)^{x \cdot \beta \cdot sk \cdot (x^* - x)}$. And obtains the following equation by taking logarithms and solving the equation:

$$x = \frac{\log\left(\frac{e(\prod (\rho_{a_i}^*)^{b_i}, g)}{e(\prod \rho_{a_i}^{b_i}, g)}\right)}{\beta \cdot sk \cdot (x^* - x)}.$$

The simulator computes all known values $(\beta, sk, x^* - x)$ to extract x and solve the DL problem. For adversary $\mathcal{A}$ to succeed in forging the authenticator, it must manipulate the unknown parameter x . However, under the DL assumption, $\mathcal{A}$ cannot compute x in polynomial time, and thus cannot attack the scheme with non-negligible probability.

Theorem 2. Our scheme can prevent SDO from obtaining file permissions by forging data or using proofs that have already passed cloud verification during the deduplication phase.

Proof. If SDO lacks a complete and correct file, it cannot generate a valid proof to pass the cloud server's verification.

- **Initialization.** $\mathcal{B}$ executes algorithm *Setup* to generates $Para = \{G_1, G_2, e, g, \{g_1, g_2, \dots, g_m\}, H, H_1, \pi_1, \pi_2\}$ and sends it to adversary $\mathcal{A}$.
- **Query Phase.** $\mathcal{A}$ selects file F and sends it to $\mathcal{B}$. $\mathcal{B}$ executes algorithm *KeyGen* algorithm to generate $K = H_1(\prod_{i=1}^{\ell} k_i)$, $sk = H_1(s^{(u)})$, and $pk = g^{sk}$, which are returns to $\mathcal{A}$. This process allows $\mathcal{A}$ to conduct multiple rounds of queries; concurrently, $\mathcal{A}$ sends file block data to $\mathcal{B}$ to request the corresponding signatures σ_i .
- **Challenge Phase.** $\mathcal{B}$ randomly selects $r_1, r_2 \in Z_p, k \in [n]$, then sends a challenge message $chal_1 = (r_1, r_2, k, ID)$ to $\mathcal{A}$. This message excludes any content from previous queries and requires $\mathcal{A}$ to return the corresponding *proof*.
- **Forgery Phase.** $\mathcal{A}$ forges an integrity proof for the file F corresponding to FID based on the challenge message sent by $\mathcal{B}$, and returns it to $\mathcal{B}$. If it passes $\mathcal{B}$'s verification, $\mathcal{A}$ wins.

Theorem 3. Our proposed scheme can achieve the verifiability and traceability of deduplication results and ensure the fairness of the deduplication process by virtue of blockchain technology.

In our scheme, integrating blockchain technology into the multi-fog-node audit architecture achieves triple security guarantees at the security level. First, the on-chain recording of the entire audit process ensures immutability and traceability. Any modification to audit evidence will break the hash chain structure of the blockchain and can be detected by all participating nodes. Second, the consensus mechanism based on threshold signatures provides Byzantine fault tolerance. The system can still reach a correct consensus even if up to $[(n-1)/3]$ fog nodes are compromised or malfunction, which eliminates the risk of single point of failure. Third, the transparency of the blockchain enables publicly verifiable auditing. Data owners can directly verify the integrity of cloud data without relying on the trustworthiness of third-party audit institutions.

Remark 1 (Resist Replay Attack). In our scheme, replay attacks may mainly occur in the audit phase and PoW phase. In the PoW phase, if adversary $\mathcal{A}$ attempts to obtain file data by eavesdropping $proof_1 = (c_{a_i}, v_j)$ during the challenge-response process, it cannot pass the cloud server's verification as it is unaware of the challenge message $chal_1 = (r_1, r_2, k)$. In the audit phase, the cloud server receives challenge information $chal_a = (z, a_i, b_i)$ from the fog node with the highest trust, generates $proof_a = (v_1, v_2, \dots, v_m, \rho)$ for verification by all fog nodes. Even if a malicious cloud server reuses the previous $proof_a^* = (v_1^*, v_2^*, \dots, v_m^*, \rho^*)$, it will fail due to inconsistency with the latest $proof_a$. Moreover, the audit time is updated in real-time and included in $chal_a$, enabling fog nodes to identify malicious cloud servers and further increasing the difficulty of replay attacks.

Remark 2 (Resist Single-Point Attack). In our scheme, the encryption key distribution process is as follows. The DOs sends $\{ID_{DO} \| H(m_1) \| \dots \| H(m_n) \| FL\}$ to each fog node $\{FN_i\}_{i=1}^{\ell}$, and the fog nodes execute the *KeyGen* algorithm to generate (K, sk, pk) for the DOs. Since the fog nodes in the scheme adopt shamir's threshold secret sharing to synthesize the secret values (K, sk, pk) , and the threshold t is a critical parameter that balances security and availability in the key generation phase. The value of t is selected based on the total number of fog nodes ℓ and the desired fault tolerance level, even if $(\ell-t)$ fog nodes fail or are dishonest, the remaining t fog nodes can still generate k_i for the DOs. This setting also prevents collusion attacks from fewer than t malicious nodes. Therefore, our scheme is resistant to single-point failures in the key generation phase. In practice, t can be adjusted according to the deployment environment and security requirements.

6. Performances analysis

In this section, we will present the performance analysis and simulation experiments of the proposed scheme. It will compare its computational, communication, and storage overheads with the deduplication and auditing schemes by Gu et al. (2024) and Zhang et al. (2024). Key notations and their descriptions are summarized in Table 5.

6.1. Functional comparison

Table 2 presents a comparison of functionality and security between our scheme and other schemes. In the scheme proposed by Gu et al. (2024), files are encrypted through conventional message-locked encryption, where the secret key is generated solely based on the content of the message itself. By contrast, in our framework, fog nodes generate encryption keys by combining their private keys with the hash block values of user messages. This mechanism is capable of resisting brute-force attacks and mitigating single point of failure. Meanwhile, the blockchain constructed among all fog nodes can further assist the

Table 3
Comparisons of user computation overhead.

Scheme	KeyGen	TagGen	Initial Upload	Subsequent Upload
Scheme (Gu et al., 2024)	$1H + 1Exp + 1Sig$	$1H + nEn$	$n(2H + 2Exp + 1En)$	$n(H + En)$
Scheme (Zhang et al., 2024)	$n(2H + 2Exp + 2Mu)$	$1H$	$n(5H + 5Exp + 3Mu + 1En)$	$(n - 1)Mu + (n + 1)En$
Our scheme	$n(t - 1)(t + 1)Mu + (nt + 1)Exp + 2H$	$(n + 1)H$	$nH + nmMu + n(m + 1)Exp$	$k(Mu + En)$

Table 4
Gas consumption of blockchain.

Blockchain	Number of challenged data blocks			
	20	40	60	80
B_c	3.51×10^3 Gas	6.15×10^3 Gas	8.72×10^3 Gas	10.46×10^3 Gas
B_v	2.56×10^5 Gas	2.64×10^5 Gas	2.83×10^5 Gas	3.12×10^5 Gas

Table 5
Notations.

Symbols	Definitions
H	Hash function
Mu	Multiplication operation
En	Encryption algorithm
Exp	Exponential operation
Sig	Signature operation
t	Threshold slice number
n	Number of blocks in F
m	Number of challenge blocks
$ Sig $	Size of the signature
$ Z_p $	Length of the element in Z_p
$ G_1 $	Group element length

cloud server in completing integrity auditing tasks. To avoid single-point failures in the key generation process, we adopt a threshold mechanism, where only t fog nodes are required to generate encryption keys for DOs. This design ensures the confidentiality of outsourced data while enabling secure and efficient key distribution. Additionally, fog nodes in our scheme jointly maintain a distributed ledger to record critical data, ensuring the reliability of deduplication and auditing results. Compared with other deduplication-auditing schemes, this scheme is more suitable for deduplication-auditing in distributed edge environments.

6.2. Performance comparison

(1). Computational Overhead. As show in Table 3, within our scheme, the data owners (DOs) interact with fog nodes to obtain $(s, \{sk_u, pk_u\}_{u=1}^t)$, based on which the DOs synthesize the encryption key K and signature key pair (sk, pk) . The overhead for the key generation phase is $n(t - 1)(t + 1)Mu + (nt + 1)Exp + 2H$. In the initial upload phase, the initial data owner (IDO) uploads the cipher-text CM and authenticator U . In our scheme, the authenticator is generated by partitioning and zoning the ciphertext, the overhead amounts to $nH + nmMu + n(m + 1)Exp$. For subsequent uploads, the repeated data owner (SDO) completes the Proof of Ownership (PoW) protocol with the cloud. This process requires prior interaction with fog nodes to obtain (K, sk, pk) , followed by generating a challenge message to return to the cloud, resulting in an overhead of $k(Mu + En)$. Additionally, the overhead for generating FL in file label generation phase is $(n + 1)H$. Analysis reveals that the authenticator generated by our scheme has the smallest size, enabling a reduction in cloud storage space.

(2). Communication Overhead. As show in Table 6, during the initial data upload phase, the initial data uploader (IDO) uploads ciphertext CM , the authenticator U , which are received via cloud communication, with a communication overhead of $2|H| + |G_1|$. In the subsequent upload phase, the subsequent data owner (SDO) interacts with the cloud to complete PoW: the cloud generates $chal_1$ and sends it to the SDO, and receives the $proof_1$ returned by the SDO, resulting in a communication overhead of $n|G_1| + 2|Z_p|$. In the *Audit.chal* process,

the cloud receives $chal_a = (z, a_i, b_i)$ from fog nodes and generates $proof_a$ return to fog nodes, with a communication overhead of $(n + m)|G_1| + 2|Z_p|$. Upon receiving the proof message, fog nodes verify $proof_a$, with a communication overhead of $n|G_1| + |Z_p|$ for the verification process.

(3). Storage Overhead. As shown in Table 7, during the data upload and audit phases, data owner (DO) do not need to store file F but need store the corresponding encryption key K and their own signature key pair (sk, pk) , resulting in a user storage overhead of $2|H| + |G_1|$. In the initial data upload phase, cloud servers need to store CM , corresponding authenticators U , and file labels FL , leading to a cloud storage overhead of $n(|Z_p| + |G_1|) + |H|$. Fog nodes, responsible for key generation and audit challenge information generation, need to store public-private key pairs (sk_u, pk_u) and audit verification results, with a storage overhead of $n|G_1| + |Z_p|$. In our scheme, distributed fog nodes are used to construct a tamper-proof distributed ledger. Comparative analysis shows that our scheme generates by partitioning message blocks m_i , while the other two schemes generate labels directly based on m_i .

(4). Blockchain Cost. To evaluate the overhead incurred by our scheme on the blockchain, we implemented the design using Solidity in the Remix integrated development environment (IDE) and deployed and tested it using the Remix VM. The main operational phases on the blockchain are audit challenge generation and proof verification; therefore, B_c denotes the overhead of the challenge generation process and B_v denotes the overhead of the audit verification. During testing, we uploaded audit challenge information of varying scales onto the blockchain, and as shown in Table 4, the Gas consumed in this step increases gradually, primarily due to the data write cost of storing indices and coefficient arrays on-chain. Subsequently, we invoked the verification function for testing, and the Gas consumption at this stage remains relatively stable, mainly stemming from the core computations required to complete the bilinear pairing verification. This indicates that the audit verification overhead of our scheme is independent of the challenge scale, demonstrating the efficiency and stability of the scheme during the verification phase; meanwhile, the linear growth of the challenge overhead aligns with the system design expectations, and the total Gas cost is within the acceptable range for public blockchains.

6.3. Experimental testing

In this section, we evaluate the performance of the proposed solution for large-scale data deduplication and integrity auditing. Experiments are conducted on a campus network-based cloud-fog testbed, consisting of ten distributed fog nodes and one central cloud server. Each fog node is equipped with an Intel Core i5-1135G7 2.4 GHz processor and 16 GB of memory. The cloud server uses an Intel Xeon Silver 4210 2.2 GHz CPU, 64 GB of memory, and 1 TB of SSD storage. The experimental results are the average values obtained from multiple experiments.

(1) Computational overhead during auditing. We compare the computational cost of three schemes under increasing concurrent users. As depicted in Fig. 3(a), audit overhead rises with user concurrency, but at different rates. Under light load (less than 100 users), our scheme maintains overhead between 15–22 ms, outperforming the Gu et al.'s scheme by about 25% and the Zhang et al.'s scheme by 40%. This gain comes from local processing at fog nodes, where audit requests are pre-verified and only summary data are sent to the blockchain. At 1000 concurrent users, the gap widens further. Our approach shifts computation to the edge, reducing cloud burden. In contrast, the Zhang

Table 6

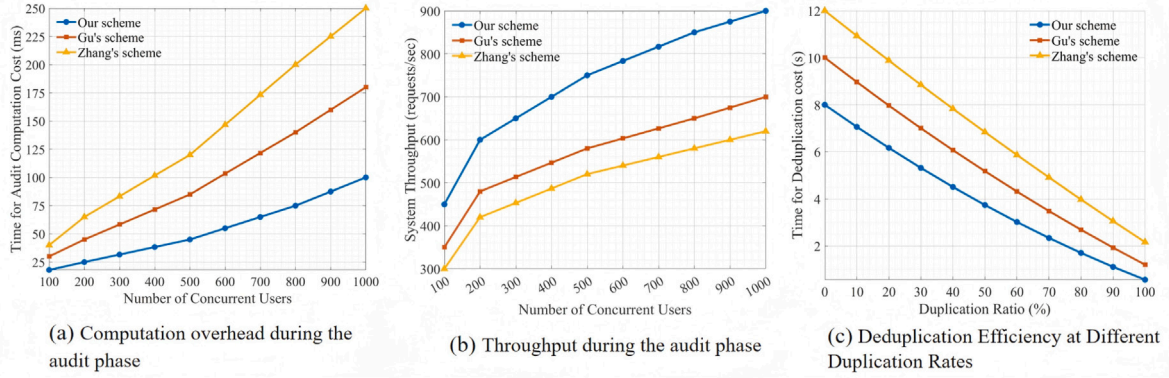
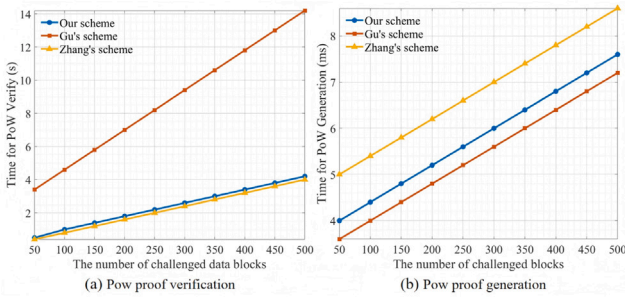
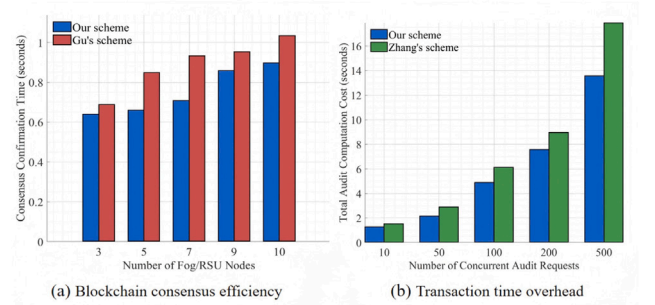
The comparisons on the communication overheads.

Scheme	Initial upload with CS	Subsequent upload with CS	Data audit with CS	Audit verify with FN
Scheme (Gu et al., 2024)	$ Z_p + 3n G_1 $	$4 Z_p + 3 G_1 + Sig $	$(4n + 9) Z_p + 4 G_1 $	$(4n + 4) G_1 + Z_p + Sig $
Scheme (Zhang et al., 2024)	$n(Z_p + H)$	$2 Z_p + G_1 $	$3 Z_p + (5 + n) G_1 $	–
Our scheme	$2 H + G_1 $	$n G_1 + 2 Z_p $	$(n + m) G_1 + 2 Z_p $	$m G_1 + Z_p $

Table 7

The comparisons on the storage overheads.

Scheme	DOs	CS	Blockchain	KS/FN
Scheme (Gu et al., 2024)	$3 Z_p $	$n G_1 + 2 Z_p $	$5n Z_p + n(m + 2) G_1 + (n + 4) G_1 $	–
Scheme (Zhang et al., 2024)	$1 Z_p $	$1 Z_p $	$(n + 3) Z_p + (n + 12) G_1 $	$n G_1 $
Our scheme	$2 H + G_1 $	$n(Z_p + G_1) + H $	$ Z_p $	$n G_1 + Z_p $

**Fig. 3.** Performance of deduplication and audit algorithms.**Fig. 4.** Computational overhead of proof of ownership.**Fig. 5.** Performance of blockchain consensus and transaction.

et al.'s scheme spends 45% of overhead on DKS key queries, which becomes the main bottleneck, while the Gu et al.'s scheme uses 35% on RSU partition coordination, limiting scalability.

(2) Audit throughput under high concurrency. Throughput for all schemes grows with user count, but saturation points differ (Fig. 3(b)). Our solution reaches a peak of 890 audits/second at around 800 users and sustains high performance even at 1000 users. The Gu et al.'s and Zhang et al.'s schemes saturate earlier. The near-linear growth of our scheme before saturation reflects effective load distribution across fog nodes. Notably, between 500–800 users, our throughput continues to climb, whereas the others decline. This scalability stems from parallel request handling at fog nodes and an on-chain batching strategy that merges multiple audit results into single transactions, lowering blockchain interaction frequency.

(3) Performance of our scheme in large-scale deduplication. We conduct experiments on a 1 TB dataset with varying duplication ratios to compare the performance of the three schemes. As illustrated in Fig. 3(c), all schemes yield the highest overhead at a 0% duplication ratio, and the overhead decreases significantly with the increase in

duplication ratio. Our scheme achieves the most substantial reduction: the overhead drops to 52.5% at a 50% duplication ratio and merely accounts for 20% of the baseline value at a 90% duplication ratio. These values are 5.0 and 12.5 percentage points lower than those of the methods proposed by Gu et al. (25%) and Zhang et al. (32.5%), respectively. In addition, our analysis of the PoW challenge generation and verification processes for the three schemes (as shown in Fig. 4) demonstrates that our solution exhibits a remarkable time efficiency advantage throughout the entire Proof of Ownership generation and verification cycle. This advantage stems from our distributed verification and multi-fog-node threshold architecture, which jointly enhance the efficiency of the entire data deduplication process.

(4) Performance of the blockchain consensus mechanism. A blockchain is constructed based on all fog nodes in our scheme, and the blockchain performance is analyzed from the perspective of the consensus mechanism by comparing with Gu et al.'s scheme. As shown in Fig. 5(a), when fog nodes scale from 3 to 10, consensus time increases linearly in both cases, but at different rates. Our scheme

consensus time grows from 0.62 s to 1.16 s, while Gu et al.'s rises from 0.75 s to 1.22 s, giving our scheme a 21.9% improvement. The advantage originates from the signature mechanism: in our design, multiple fog nodes can produce partial signatures concurrently, which are later aggregated by any node; Gu et al.'s scheme must collect votes sequentially from each RSU partition, adding coordination delays. Moreover, our approach shows more gradual performance degradation as nodes increase, offering better scalability while tolerating up to one-third node failures without compromising security.

(5) Computational Overhead of the Blockchain. To evaluate the blockchain-based distributed audit in our scheme, we conduct a comparison with the centralized third-party audit in the scheme proposed by Zhang et al. As shown in Fig. 5(b), the overhead of the two schemes is comparable under low concurrency, with Zhang et al.'s scheme even being slightly lower. However, as the concurrency increases, the overhead of Zhang et al.'s scheme is 31.1% higher than that of our scheme. A breakdown analysis reveals that 65% of our overhead is attributed to the computation of distributed fog nodes, which leverages parallel edge resources. In Zhang et al.'s scheme, key queries have to be processed through limited Distributed Key Servers (DKS), which creates a central bottleneck. In the simulation environment, the impact of consensus delay among fog nodes on end-to-end audit response time is less than 5%. Although our solution incurs 28% of blockchain-related overhead, it eliminates the trust dependency on third-party audits and enables a fully decentralized audit. Furthermore, traditional methods generate additional costs for certificate management and third-party audits, and their long-term operational costs may exceed the blockchain-based overhead even at an equivalent security level.

7. Conclusions

In this paper, we propose a fog-assisted integrated scheme to address the separation of data deduplication and integrity auditing in traditional cloud-fog environment, enabling their collaborative implementation. Compared with existing deduplication schemes, this method effectively mitigates the inherent security vulnerabilities of traditional message-locked encryption (MLE). Multiple fog nodes utilize threshold technology to generate encryption and authentication keys, ensuring the correctness and efficiency of the deduplication function. Moreover, the blockchain maintained by fog nodes is leveraged to perform audit tasks, guaranteeing the correctness, transparency, and immutability of the audit process. Security and performance analysis results confirm that the system ensures security in both deduplication and auditing processes, with significant performance advantages. It provides a practical solution for the collaborative implementation of data deduplication and auditing in cloud storage, and offers valuable reference for data security management in distributed environments.

CRedit authorship contribution statement

Siqi Wu: Writing – original draft, Visualization, Validation, Formal analysis. **Jinyong Chang:** Writing – review & editing, Supervision. **Yanyan Ji:** Resources. **Yifan Dong:** Software. **Siyi Wang:** Formal analysis.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported in part by the National Natural Science Foundation of China under Grant 62476212, in part by the Key Research and Development Program Project of Shaanxi Province under Grant 2025CYYBX-007, and in part by the Natural Science Foundation Project of Shaanxi Province under Grant 2025JCYS-774.

Data availability

The data that has been used is confidential.

References

- Alqarni, A.K., Fei, Z., 2026. A space-efficient shredding-based method for secure data deletion in multi-cloud environments. In: International Conference on Computing, Networking and Communications. ICNC, pp. 655–659.
- Armbrust, M., Fox, A., Griffith, R., et al., 2010. A view of cloud computing. *Commun. ACM* 53 (4), 50–58.
- Aroulanandam, V.V., Krishna, K.J., Babu, A.V., et al., 2024. An efficient methodology to audit data in cloud storage to ensure reliability and integrity using secured crypto schemes. In: 2024 International Conference on Innovative Computing, Intelligent Communication and Smart Electrical Systems. pp. 1–6.
- Ateniese, G., Burns, R., Curtmola, R., et al., 2007. Provable data possession at untrusted stores. In: Proceedings of the 14th ACM Conference on Computer and Communications Security. pp. 598–609.
- Bellare, M., Keelveedhi, S., Ristenpart, T., 2013. Message-locked encryption and secure deduplication. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. Vol. 7881, pp. 296–312.
- Douceur, J.R., Adya, A., Bolosky, W.J., et al., 2002. Reclaiming space from duplicate files in a serverless distributed file system. In: Proceedings 22nd International Conference on Distributed Computing Systems. pp. 617–624.
- Gu, K., Wang, Y., Qiu, J., et al., 2024. Blockchain-based data deduplication and distributed audit for shared data in cloud-fog computing-based VANETs. *IEEE Trans. Netw. Serv. Manag.* 21 (5), 5548–5565.
- Ha, G., Chen, Y., Cai, Z., et al., 2025. Random coding responses for resisting side-channel attacks in client-side deduplicated cloud storage. *IEEE Trans. Serv. Comput.* 18 (3), 1697–1710.
- Halevi, S., Harnik, D., Pinkas, B., et al., 2011. Proofs of ownership in remote storage systems. In: Proceedings of the 18th ACM Conference on Computer and Communications Security. Vol. 7881, pp. 491–500.
- He, Z., Tian, W., Xie, Z., et al., 2026. Smart-to-compress: A predictive and game-theoretic framework for data reduction decisions. *IEEE Trans. Cloud Comput.* 14 (1), 406–418.
- Hertogh, M., Quakkelaar, D., Raymakers, T., et al., 2026. Rain: Transiently leaking data from public clouds using old vulnerabilities. In: IEEE Symposium on Security and Privacy. SP, pp. 1172–1188.
- Juels, A., Kaliski, Jr., B.S., 2007. PORs: Proofs of retrievability for large files. In: Proceedings of the 14th ACM Conference on Computer and Communications Security. pp. 584–597.
- Liu, Y., 2025. The latest application and security analysis of cryptography in cloud storage data audit. *Procedia Comput. Sci.* 259, 984–990.
- Luo, R., He, Q., Chen, F., et al., 2025. Ripple: Enabling decentralized data deduplication at the edge. *IEEE Trans. Parallel Distrib. Syst.* 36 (1), 55–66.
- Miao, Y., Gai, K., Zhu, L., et al., 2023. Blockchain-based shared data integrity auditing and deduplication. *IEEE Trans. Dependable Secur. Comput.* 21 (4), 3688–3703.
- Peng, X., Shen, W., Yang, Y., et al., 2025. Secure deduplication and cloud storage auditing with efficient dynamic ownership management and data dynamics. *IEEE Trans. Netw. Serv. Manag.* 22 (4), 3463–3479.
- Rizvi, S., Razaque, A., Cover, K., 2015. Third-party auditor (TPA): A potential solution for securing a cloud environment. In: 2015 IEEE 2nd International Conference on Cyber Security and Cloud Computing. pp. 31–36.
- Swapnali, M., Sangita, C., 2016. Third party public auditing scheme for cloud storage. *Procedia Comput. Sci.* 79, 69–76.
- Tian, H., Chen, Y., Chang, C., et al., 2017. Dynamic-hash-table based public auditing for secure cloud storage. *IEEE Trans. Serv. Comput.* 10 (5), 701–714.
- Tian, G., Hu, Y., Wei, J., et al., 2022. Blockchain-based secure deduplication and shared auditing in decentralized storage. *IEEE Trans. Dependable Secur. Comput.* 19 (6), 3941–3954.
- Wang, B., Li, B., Li, H., 2014. Oruta: Privacy-preserving public auditing for shared data in the cloud. *IEEE Trans. Cloud Comput.* 2 (1), 43–56.
- Wang, P., Liu, R., Huang, K., et al., 2026. iRUC: Reducing inter-microservice data communication in data-intensive systems via unified computation. *IEEE Trans. Softw. Eng.* 1–16.
- Wang, H., Qin, H., Zhao, M., et al., 2020. Blockchain-based fair payment smart contract for public cloud storage auditing. *Inform. Sci.* 348–362.
- Xu, Y., Cheng, H., Zheng, J., et al., 2026. MPA: Lightweight and updatable integrity auditing for decentralized storage using Merkle trees and polynomial commitments. *IEEE Trans. Inf. Forensics Secur.* 21, 2162–2176.
- Yang, X., Li, R., Li, S., et al., 2026. Shared data integrity audit scheme based on CP-ABE and deduplication for cloud storage. *IEEE Trans. Netw. Serv. Manag.* 23, 1615–1629.
- Yu, J., Jun, Y., 2024. Multi-copy edge data integrity verification supporting dynamics and batch auditing. *Procedia Comput. Sci.* 247, 1285–1294.

- Yuan, J., Yu, S., 2013. Secure and constant cost public cloud storage auditing with deduplication. In: 2013 IEEE Conference on Communications and Network Security. pp. 145–153.
- Zhang, G., Chen, P., Liao, Y., 2024. Blockchain-based secure and verifiable deduplication scheme for cloud-assisted internet of things. *IEEE Internet Things J.* 11 (8), 13995–14006.
- Zhang, Q., Qian, S., Cui, J., et al., 2025. Blockchain-based privacy-preserving deduplication and integrity auditing in cloud storage. *IEEE Trans. Comput.* 74 (5), 1717–1729.
- Zheng, X., Shen, W., Su, Y., et al., 2025. DIADD: Secure deduplication and efficient data integrity auditing with data dynamics for cloud storage. *IEEE Trans. Netw. Serv. Manag.* 22 (1), 299–316.